AGENT EVOLUTION · NOTEBOOK 3 OF 8

Version 3 — Semantic Memory: Pinecone Knowledge Base + PostgreSQL

A guide to `agent_pinecone_semantic_memory.ipynb`: why the agent needs knowledge before it writes SQL, how semantic search finds it, what each lookup tool returns, and how the agent combines memory with hands.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · Bedrock chat model · OpenAI embeddings · Pinecone · PostgreSQL

Tools: `lookup_schema` · `lookup_examples` · `lookup_common` · `run_query`

1. What changes from Version 2
2. The end-to-end workflow
3. Memory and hands
4. Why semantic search
5. The knowledge base
6. Inside a lookup
7. The three lookup tools
8. The loop with retrieval
9. One call, step by step
10. Using examples safely
11. What changed in the prompt
12. Testing and tuning retrieval
13. The limitation, and what Version 4 changes
14. Key takeaways

1 What changes from Version 2

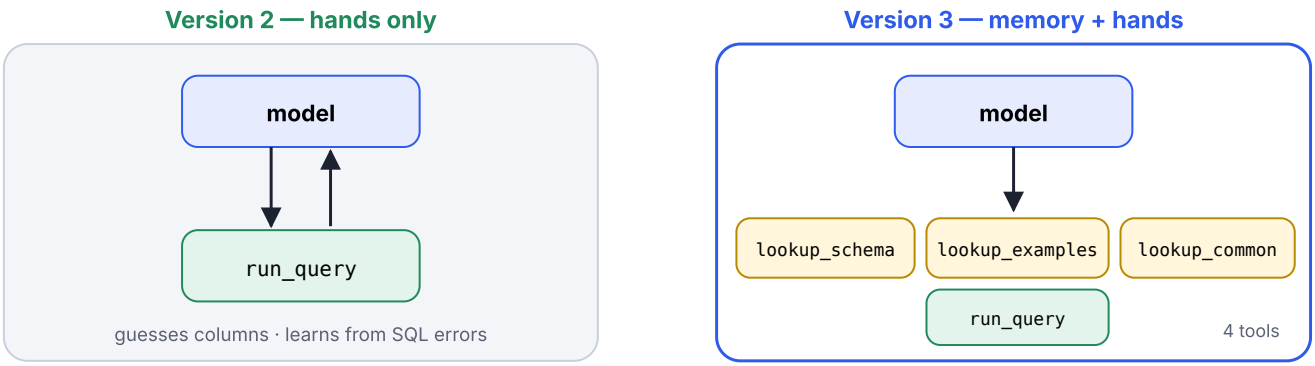


Figure 1. Version 3 adds three lookup tools. `run_query` is unchanged.

	Version 2	Version 3
How it knows the schema	table names in the prompt; guesses columns	`lookup_schema` → Pinecone `nlq-schema`
Similar solved questions	—	`lookup_examples` → `nlq-examples`
Business definitions	—	`lookup_common` → `common`
Table list in the prompt	yes (14 names)	**removed** — the agent looks it up

Chat model	OpenAI `gpt-6-astra`	AWS Bedrock (`BEDROCK_CHAT_MODEL_ID`)
OpenAI is used for	the chat model	query embeddings only
`run_query`	first-word gate, read-only, 50 rows	unchanged

The OpenAI embedding model stays because the vectors in Pinecone were built with it. A search phrase must be turned into a vector by the **same** model, or the comparison is meaningless (section 5).

2 The end-to-end workflow

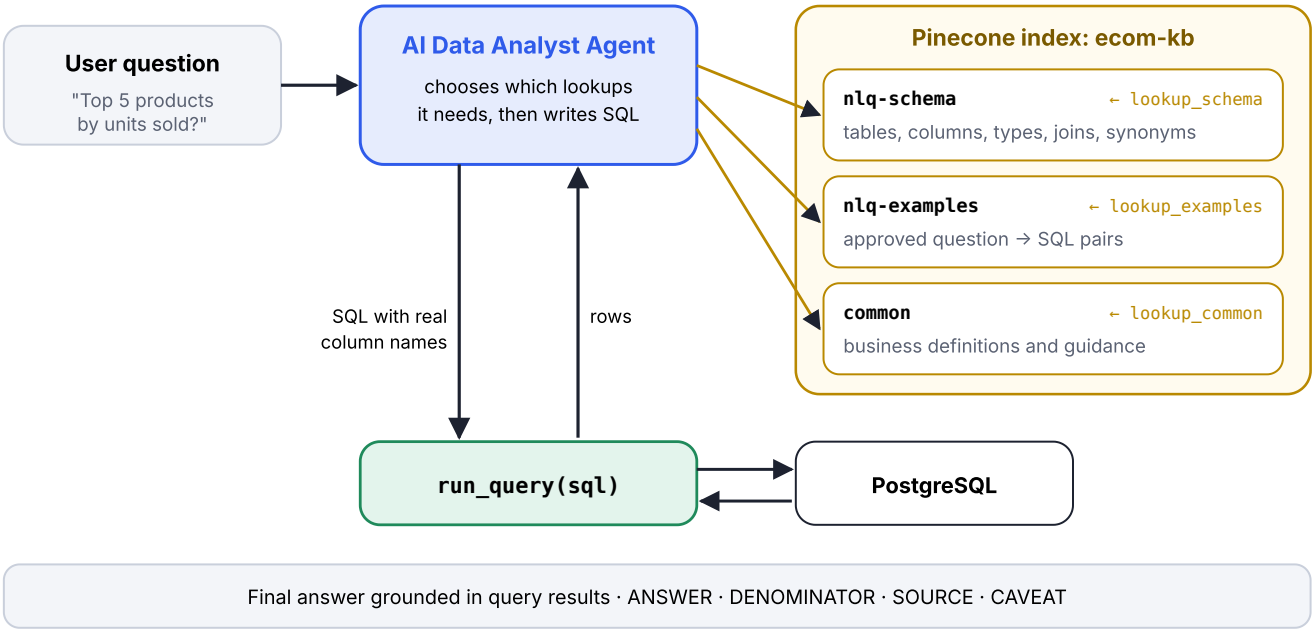


Figure 2. Version 3 end to end. Lookups read knowledge from Pinecone; `run_query` reads data from PostgreSQL.

1. **The question arrives.** "Top 5 products by units sold?"
2. **The model decides what it does not know yet.** Which table holds quantities? How do order lines join to products? Has a similar question been solved?
3. **It calls the lookups it needs**, for example `lookup_schema("units sold per product")` and `lookup_examples("top products by units sold")`.
4. **It writes SQL with real column names** from what it retrieved, and calls `run_query`.
5. **It answers from the rows**, in the same four sections as Version 2.

> The agent does **not** call every lookup every time. The prompt says "choose those relevant to the question". A question about a definition may need only `lookup_common`; a simple count may need only `lookup_schema`.

3 Memory and hands

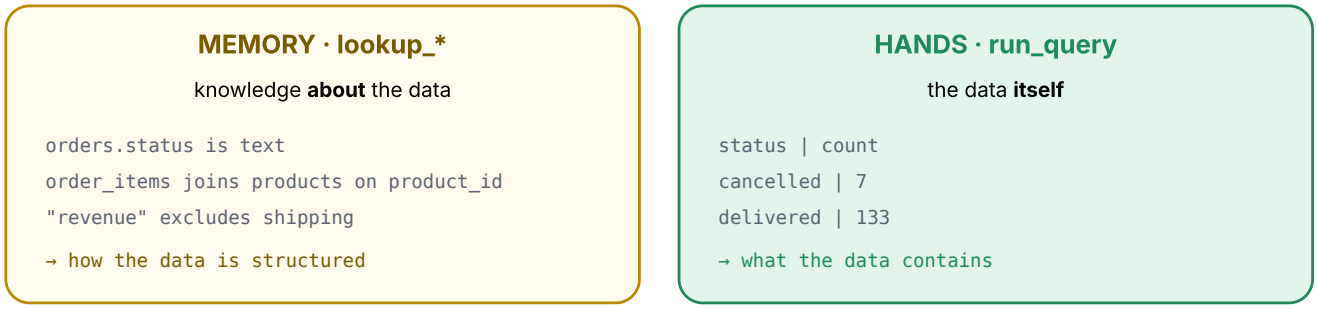


Figure 3. Two kinds of retrieval. Example values are illustrative.

Part	Tools	Answers
Brain	the chat model	which tool next? what does this result mean?
Memory	`lookup_schema`, `lookup_examples`, `lookup_common`	what are the tables and columns called, how do they join, what do business terms mean?
Hands	`run_query`	what is actually in the database right now?

Lookups never touch PostgreSQL, and `run_query` never touches Pinecone. Each tool docstring says so explicitly (“Retrieves metadata only; does not execute SQL”), so the model does not expect data from a lookup.

4 Why semantic search

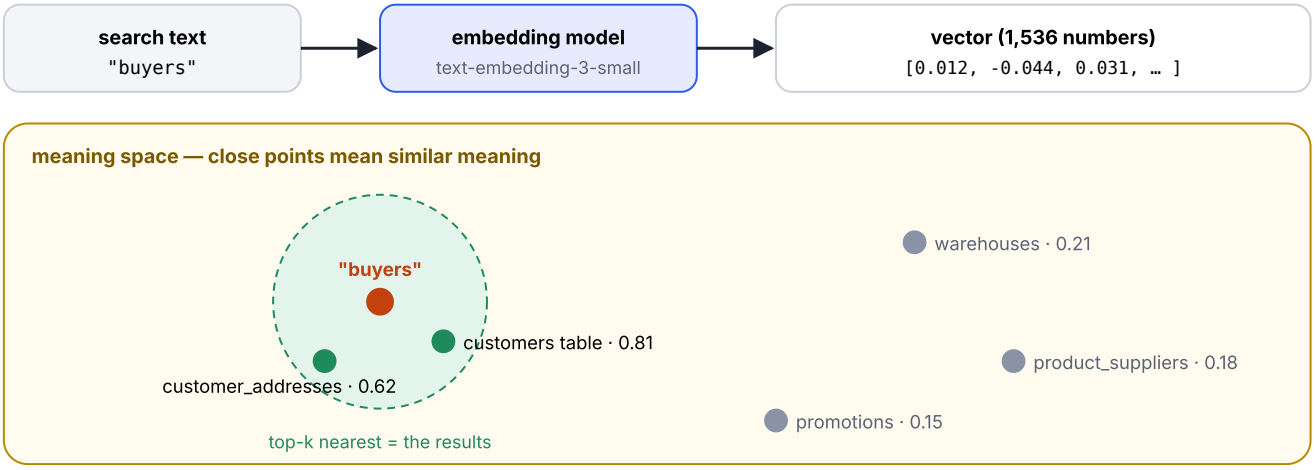


Figure 4. Semantic search. The search text becomes a vector; the nearest stored vectors are returned. Scores are illustrative.

A user says “buyers”. The schema calls them `customers`. A keyword search for “buyers” finds nothing, because the word is not in the schema text. A semantic search finds `customers`, because the two words mean nearly the same thing.

1. An **embedding model** turns text into a list of numbers (a **vector**). `text-embedding-3-small` produces 1,536 numbers per text.
2. Texts with similar meaning get vectors that point in similar directions.
3. Pinecone compares the search vector with every stored vector and returns the closest ones, with a **score**. Higher means closer.

User says	Schema or KB says	Keyword match?	Semantic match?
buyers	`customers`	No	Yes
money kept	revenue net of refunds, by payment status	No	Yes
out of stock	`inventory` quantity = 0	No	Yes
best-selling	an approved “top products by units sold” example	Partly	Yes

> A score measures **similarity**, not correctness. The top result is the closest text, not necessarily the right answer. That is why the model still reads the results and decides what to use.

5 The knowledge base

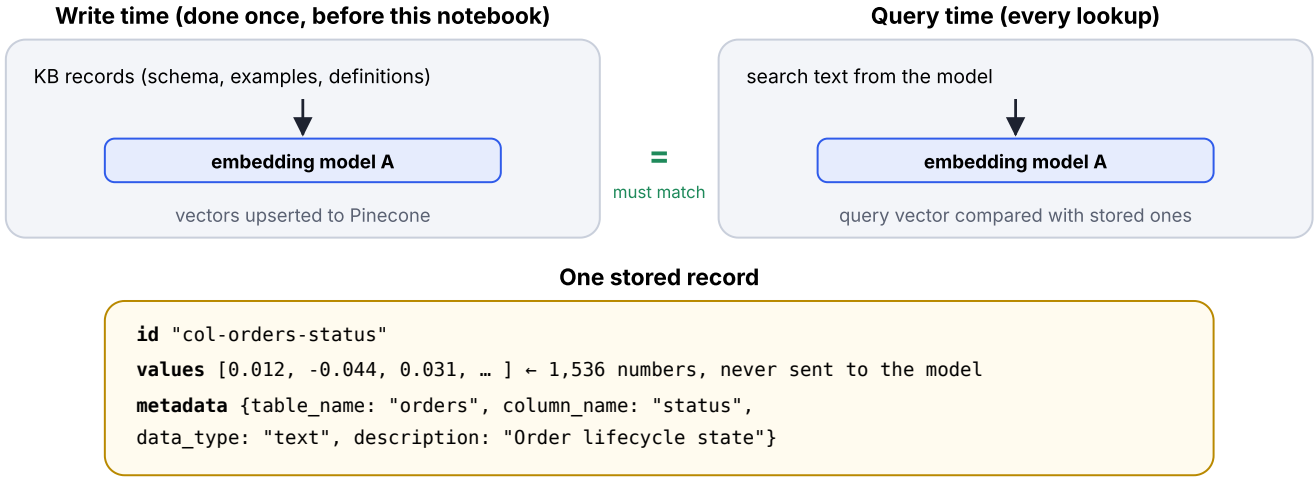


Figure 5. Write time and query time must use the same embedding model. Record fields are illustrative.

The notebook **reads** an existing Pinecone index called `ecom-kb`. It does not build or upload anything.

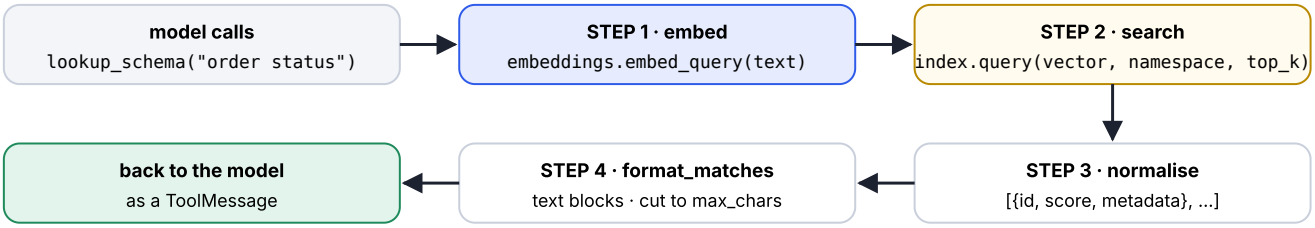
Namespace	Holds	Read by
`nlq-schema`	table and column descriptions, types, joins, synonyms	`lookup_schema(term)`
`nlq-examples`	approved questions with their SQL	`lookup_examples(question)`
`common`	business definitions and guidance	`lookup_common(question)`

A **namespace** is a separate partition inside one index. A query searches one namespace only, so schema records never compete with example records for the top places.

Each record has an **id**, a **vector** and **metadata**. The vector is used only for matching. The metadata is the readable text the model gets back.

> **The embedding model must match.** If the stored vectors came from model A and the query uses model B, two things can happen. If the vector sizes differ, Pinecone rejects the query with a dimension error. If the sizes match, the query runs and returns **meaningless** results with no error at all. The second case is worse because nothing tells you it is wrong.

6 Inside a lookup




```
RESULT 1 | id=col-orders-status | score=0.8312
table_name: orders
column_name: status
description: Order lifecycle state
--- RESULT 2 | id=tbl-orders | score=0.7904 …
```

real format_matches output, example metadata

Figure 6. One lookup. All three tools share `search_namespace` and `format_matches`; only the namespace and limits differ.

1. **Embed.** The search text becomes a 1,536-number vector.
2. **Search.** Pinecone returns the `top_k` closest records in one namespace, with their metadata.
3. **Normalise.** The Pinecone client may return a dict or an object; `search_namespace` reads both and produces a plain list of `{id, score, metadata}`.
4. **Format.** `format_matches` writes each match as a `RESULT n | id=… | score=…` line followed by one line per metadata field. Lists and dicts become JSON text. The whole output is cut at `max_chars`, with `...[truncated]` added if it was cut.

What the model gets: ids, scores and readable metadata. The ids let you trace an answer back to the exact knowledge-base records it used.

What the model never gets: the vectors. 1,536 numbers per record would waste thousands of tokens and mean nothing to the model.

If nothing matches, the tool returns `No matching knowledge-base records were found.`, which is text the model can act on, just like an empty SQL result.

7 The three lookup tools

Tool	Argument	Namespace	top_k	max_chars	Docstring tells the model
`lookup_schema`	`term`	`nlq-schema`	8	12,000	confirm real table/column names, types, descriptions, synonyms, business meaning
`lookup_examples`	`question`	`nlq-examples`	4	10,000	find approved NL-to-SQL examples; use them as patterns, then verify with `lookup_schema`
`lookup_common`	`question`	`common`	4	8,000	find shared business definitions or guidance

Why different settings

- **`lookup_schema` takes a `term`**, a short phrase like “order status”. Schema records are short, so it returns more of them (8).

- `lookup_examples` **takes a whole** `question`, because it matches against stored questions. Each example carries SQL, so 4 is enough.
- `max_chars` caps how much text one lookup can add to the conversation. Every lookup result stays in the message list and is resent on every later pass.

> **The argument name is part of the instructions.** The model sees `term` for one tool and `question` for the other two, so it sends a short phrase to the first and a full question to the others. These schemas were checked against the notebook's real tools.

8 The loop with retrieval

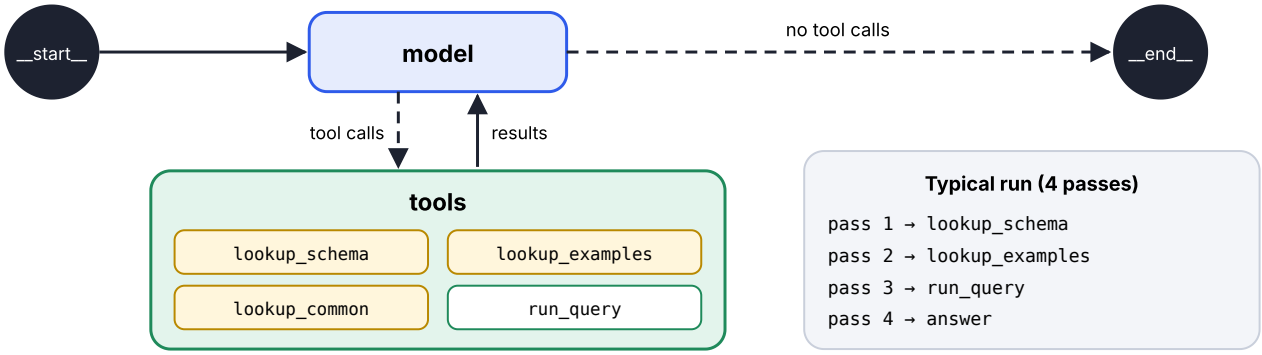


Figure 7. The graph is the same four nodes as Version 2. Only the contents of the `tools` node changed.

`agent.get_graph().nodes` is still `['__start__', 'model', 'tools', '__end__']`. Adding tools does not add nodes. The one `tools` node runs whichever tool the model asks for.

Run	Passes	Notes
schema lookup → query → answer	3	the shortest data run
schema + example lookups → query → answer	4	typical for “top N” or ranking questions
two lookups requested in one reply → query → answer	3	the model may ask for two tools at once; both results come back in the same step
definition question (e.g. “how is revenue defined?”)	2	`lookup_common` only, no SQL needed

> Lookups add passes, but they replace the **guess → SQL error → retry** passes from Version 2. A lookup is cheaper and safer than a failed query: it never touches the database and its answer is designed to be read.

9 One call, step by step

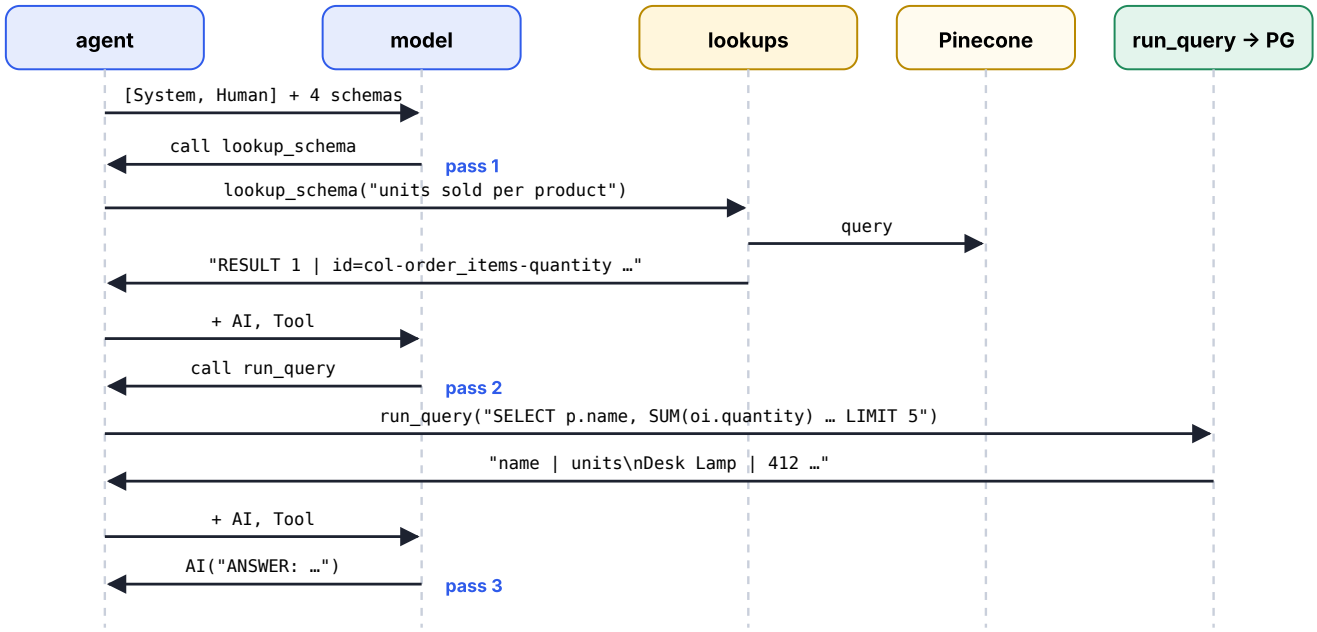


Figure 8. A three-pass run: one lookup, one query, one answer. Values are illustrative.

```
result["messages"] == [
  HumanMessage("Top 5 products by units sold?"),
  AIMessage(tool_calls=[{"name": "lookup_schema",
                         "args": {"term": "units sold per product"}, "id": "call0"}]),
  ToolMessage("RESULT 1 | id=col-order_items-quantity | score=0.84\n...",
              tool_call_id="call0", name="lookup_schema"),
  AIMessage(tool_calls=[{"name": "run_query",
                         "args": {"sql": "SELECT p.name, SUM(oi.quantity) AS units ..."},
                         "id": "call1"}]),
  ToolMessage("name | units\nDesk Lamp | 412\n...\n(5 rows.)",
              tool_call_id="call1", name="run_query"),
  AIMessage("ANSWER: The top 5 products by units sold are ...")
]
```

Same message types as Version 2. The difference is in the order: **knowledge first, data second**. The lookup result stays in the message list, so on pass 2 the model writes SQL while it can still “see” the real column names.

To trace an answer back to its sources, read the ids in the `lookup_*` observations. Version 8 collects them automatically as `grounding_record_ids`.

10 Using examples safely

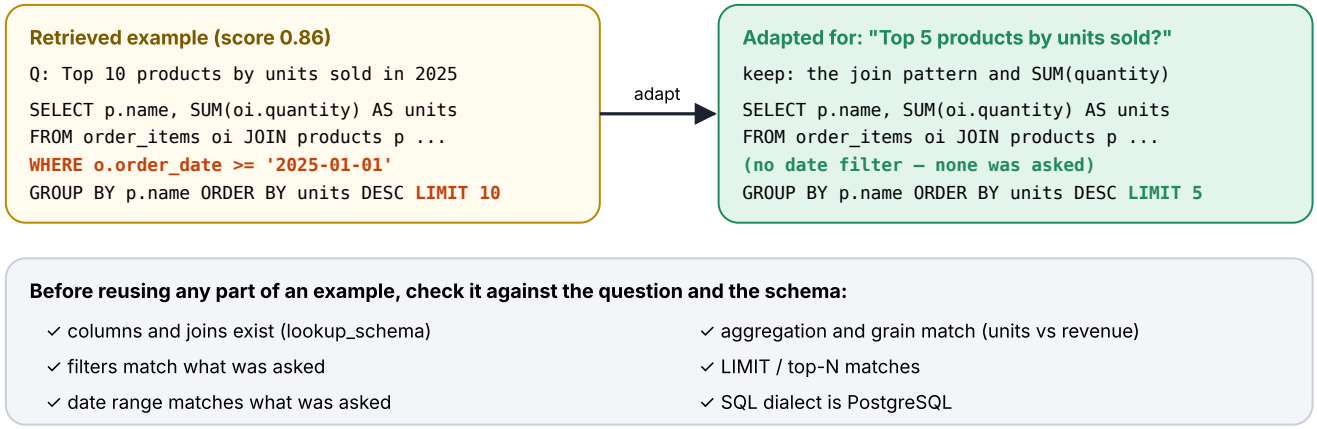


Figure 9. An example is a pattern. Keep the structure; change everything that differs from the question. Values are illustrative.

Examples are the most powerful lookup and the easiest to misuse. A close match can still differ in the details that matter.

Copying the example blindly would...	Result
keep `WHERE order_date >= '2025-01-01'`	silently drops older orders the user did not exclude
keep `LIMIT 10`	returns 10 rows when 5 were asked
use `SUM(line_total)` from a "top by revenue" example	ranks by money, not units
use another dialect's syntax (e.g. `TOP 5`)	SQL error in PostgreSQL

The prompt covers this directly: "Treat retrieved examples as patterns, not guaranteed-correct answers. Verify columns, joins, filters, date ranges, and aggregation against retrieved schema." The `lookup_examples` docstring repeats it: "use examples as patterns, then verify schema details using `lookup_schema`".

11 What changed in the prompt

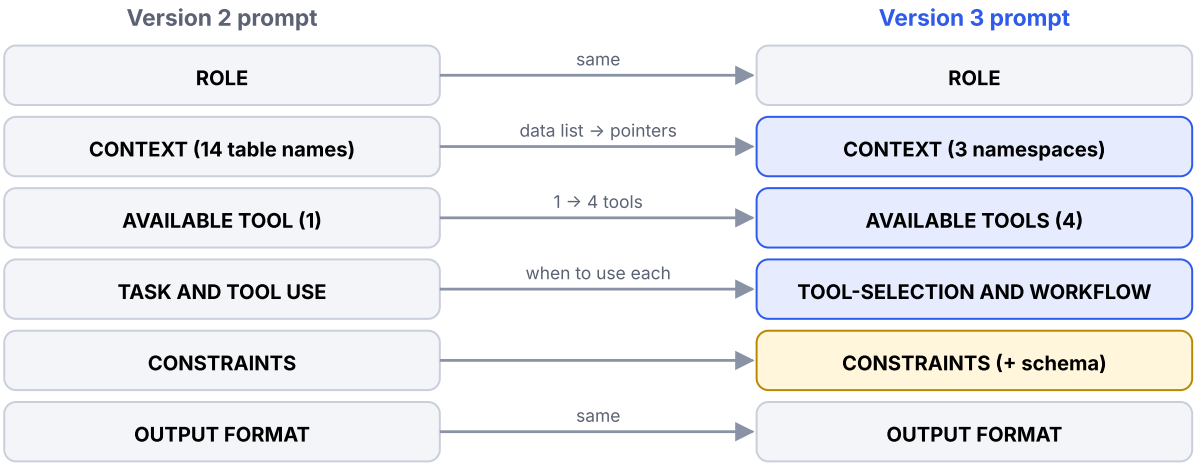


Figure 10. Blue blocks are rewritten; amber gained a rule. Same anatomy as Versions 1 and 2.

Block	What changed
CONTEXT	The 14 table names are **gone**. It now describes the three namespaces and says PostgreSQL is the source of truth for data.
AVAILABLE TOOLS	Four tools, each with its namespace. Says plainly: "This version does not have a dedicated SQL validation tool."
TOOL-SELECTION AND WORKFLOW	When to use each lookup; "do not call every retrieval tool automatically"; examples are patterns; one read-only query, then inspect the result; retry on error.
CONSTRAINTS	Adds "never invent ... schema details or business definitions".

Why the table list was removed. A list written into the prompt is data, and data in a prompt goes stale: add a table and the prompt is wrong until someone edits it. With lookups, the agent asks the knowledge base, so a schema change only needs a knowledge-base update. Facts live in the data store; the prompt only says where to find them.

Each tool is described twice: once in its docstring (what it does), once in the prompt (when to use it in this job). Keep the two consistent. If they disagree, the model gets mixed instructions.

12 Testing and tuning retrieval

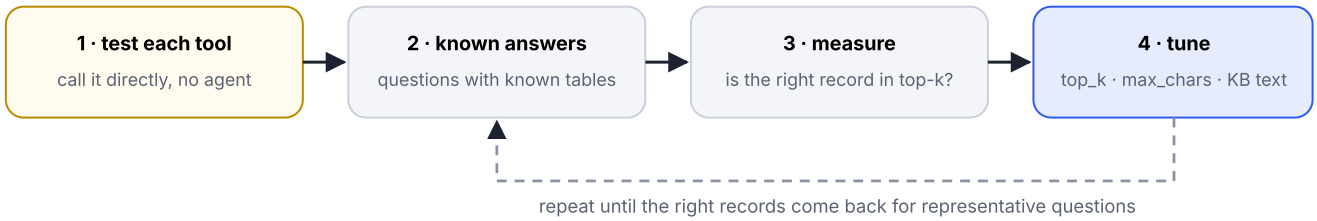


Figure 11. Test retrieval on its own before judging the agent.

The notebook calls each lookup directly before running the agent:

```
print(lookup_schema.invoke({"term": "product category identifier"}))
print(lookup_examples.invoke({"question": "top products by units sold"}))
print(lookup_common.invoke({"question": "how should revenue be calculated?"}))
```

This separates two questions. **Does the knowledge base return the right records?** And then: **does the agent use them well?** If retrieval is wrong, no prompt change will fix the agent.

What to look at

Signal	What it suggests
The right record is RESULT 1 with a clear score gap	retrieval is working for this phrase
The right record is present but ranked 6th	`top_k` is just large enough; the record's description may need better wording or synonyms
The right record is missing	add synonyms or clearer descriptions to the KB record, or raise `top_k`
Output ends with `...[truncated]`	`max_chars` cut real content; raise it or trim metadata
All scores are low and close together	nothing in the KB matches well; the agent should say what is missing

> **There is no score threshold in the code.** The tools always return the top-k records, even when the best match is weak. The model sees the scores and must judge them itself. The values 8 / 4 / 4 and the `max_chars` limits are starting points: test them with real questions before relying on them.

13 The limitation, and what Version 4 changes

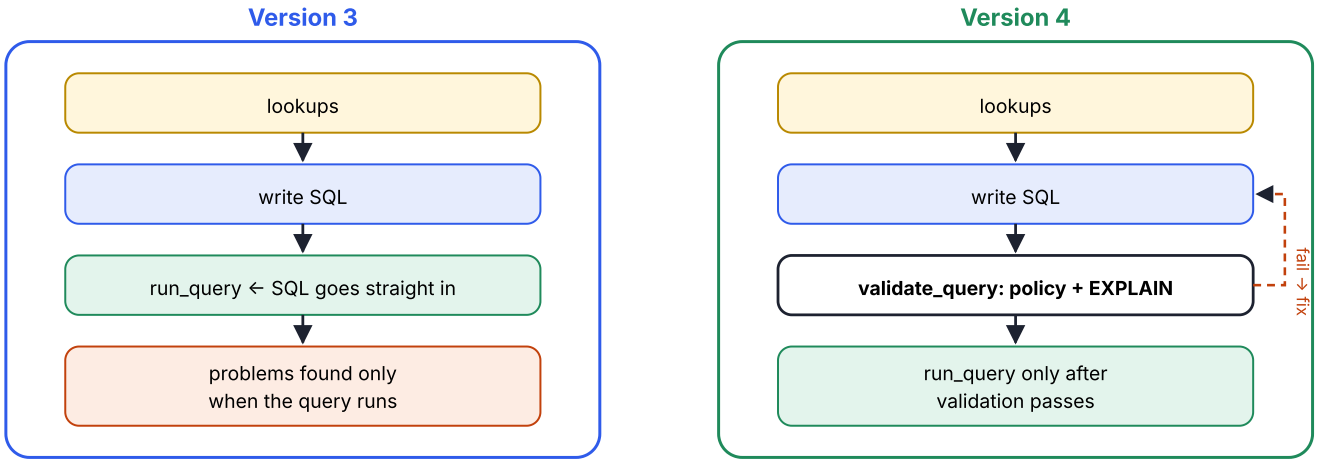


Figure 12. Version 3 knows the schema but still sends SQL straight to the database. Version 4 checks it first.

Version 3 fixes the knowledge problem. The agent now writes SQL with real names. But nothing checks that SQL before it runs:

- **The safety check is still the first-word gate.** A write hidden in a `WITH`, or after a `;`, passes it. Only the read-only connection stops it.
- **Mistakes are found by running them.** A wrong join or a typo becomes a real database call that fails.
- **Correct names do not mean correct SQL.** The model can still get a type wrong, misuse a function, or mix dialects from an example.

	Version 3	Version 4
Check before running	first word only	`validate_query`: SQL policy + PostgreSQL `EXPLAIN`
Multiple statements / hidden writes	not detected by the tool	rejected
Unknown column, type error	found when the query runs	found by `EXPLAIN`, without running it
`run_query` check	first word	the same policy as `validate_query`

14 Key takeaways

1. **Memory and hands are different tools.** Lookups return knowledge about the data; `run_query` returns the data. Neither does the other's job.
2. **Semantic search matches meaning, not words.** “buyers” finds `customers`. A score is similarity, not proof of correctness.
3. **The embedding model must match the one used to build the index.** A mismatch can fail loudly (dimension error) or silently (wrong results).
4. **One index, three namespaces.** Schema, examples and definitions are searched separately so they do not crowd each other out.
5. **The model gets ids, scores and metadata, never vectors.** The ids let you trace an answer to its sources.
6. **The graph does not change when tools are added.** Still four nodes; the `tools` node runs whichever tool is asked for.
7. **Knowledge first, data second.** Lookup results stay in the messages, so the SQL is written with real names in view.
8. **Examples are patterns.** Keep the structure; check columns, filters, dates, aggregation, limits and dialect.
9. **Facts belong in the data store, not the prompt.** The table list was removed; the prompt now points to where facts live.
10. **Test retrieval on its own** with known-answer questions before judging the agent.

> **Next — Version 4:** `validate_query(sql)` checks every query with a SQL policy and PostgreSQL `EXPLAIN` before `run_query` is allowed to run it.